

Merge Sort

Divide, Conquer and Merge

1 What is Merge Sort?

Merge Sort is a sorting algorithm based on the:

Divide and Conquer

strategy.

The main idea is:

1. **Divide** the array into two halves.
2. **Conquer** by recursively sorting the two halves.
3. **Merge** the two sorted halves.

The important idea is:

Divide $\rightarrow$ Sort $\rightarrow$ Merge

2 Basic Idea

Suppose we have:

[38, 27, 43, 3, 9, 82, 10]

Instead of sorting all elements together, Merge Sort divides the array into smaller parts.

For example:

[38, 27, 43, 3, 9, 82, 10]

is divided into:

[38, 27, 43, 3] [9, 82, 10]

These parts are again divided:

[38, 27] [43, 3]

and:

[9, 82] [10]

The division continues until every part contains only one element.

A single element is already sorted.

Then the small sorted arrays are merged to form larger sorted arrays.

3 How to Divide the Array?

For Merge Sort, we use three important variables:

$$\boxed{low, \quad mid, \quad high}$$

Here:

- low = starting index of the current array.
- $high$ = ending index of the current array.
- mid = middle index used to divide the array.

The middle is calculated as:

$$\boxed{mid = \left\lfloor \frac{low + high}{2} \right\rfloor}$$

The array is divided into:

$$[low \dots mid]$$

and:

$$[mid + 1 \dots high]$$

Therefore:

$$\boxed{\text{Left part} = low \text{ to } mid}$$

$$\boxed{\text{Right part} = mid + 1 \text{ to } high}$$

4 How Does the Value of n Decide the Division?

Suppose the array has n elements.

Initially:

$$low = 0$$

and:

$$high = n - 1$$

Therefore:

$$mid = \left\lfloor \frac{0 + (n - 1)}{2} \right\rfloor$$

The array is divided approximately into two equal parts.

Case 1: $n = 8$

For:

$$n = 8$$

we have:

$$low = 0, \quad high = 7$$

Therefore:

$$mid = \left\lfloor \frac{0 + 7}{2} \right\rfloor = 3$$

So:

$$[0 \dots 3] \quad [4 \dots 7]$$

Each part contains:

4 elements

Then:

$$4 \rightarrow 2 \rightarrow 1$$

So the division is:

$$8 \rightarrow 4 + 4 \rightarrow 2 + 2 + 2 + 2 \rightarrow 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$$

Case 2: $n = 7$

For:

$$n = 7$$

we have:

$$low = 0, \quad high = 6$$

Therefore:

$$mid = \left\lfloor \frac{0 + 6}{2} \right\rfloor = 3$$

So the array is divided into:

$$[0 \dots 3] \quad [4 \dots 6]$$

The sizes are:

$$4 + 3$$

Then:

$$4 \rightarrow 2 + 2$$

and:

$$3 \rightarrow 2 + 1$$

Finally, every part contains one element.

Therefore, even when n is odd, Merge Sort divides the array into two approximately equal parts.

5 Important Base Condition

The division stops when:

$$low \geq high$$

This means that the current part contains zero or one element.

For one element:

$$low = high$$

For example:

$$[8]$$

is already sorted.

Therefore:

If $low \geq high$, stop dividing.

6 Merge Sort Algorithm

The main Merge Sort algorithm is:

```
1 MergeSort(A, low, high)
2
3 if low < high
4     mid = (low + high) / 2
5
6     MergeSort(A, low, mid)
7
8     MergeSort(A, mid + 1, high)
9
10    Merge(A, low, mid, high)
```

The important point is:

MergeSort(left) $\rightarrow$ MergeSort(right) $\rightarrow$ Merge

7 Meaning of Low, Mid and High

Consider:

$$A = [38, 27, 43, 3, 9, 82, 10, 15]$$

Initially:

$$low = 0$$

$$high = 7$$

Therefore:

$$mid = \frac{0 + 7}{2} = 3$$

The array is divided as:

$$\underbrace{[38, 27, 43, 3]}_{low=0, mid=3} \quad \underbrace{[9, 82, 10, 15]}_{mid+1=4, high=7}$$

Therefore:

$$\boxed{\text{Left} = A[low \dots mid]}$$

$$\boxed{\text{Right} = A[mid + 1 \dots high]}$$

8 Merging Two Sorted Arrays

After the division is complete, we start merging.

Suppose we have two sorted parts:

$$[12, 27, 38]$$

and:

$$[15, 24, 43]$$

We compare the first elements of both parts.

$$12 < 15$$

So we copy 12.

Then compare:

$$27 \text{ and } 15$$

Since:

$$15 < 27$$

copy 15.

Continue this process until one part becomes empty.
The remaining elements of the other part are then copied.
Final result:

[12, 15, 24, 27, 38, 43]

9 Using Infinity During Merging

A common way to explain the Merge operation is by adding a special value:

∞

at the end of each temporary sorted array.
Suppose:

$L = [12, 27, 38]$

and:

$R = [15, 24, 43]$

We add ∞ at the end:

$L = [12, 27, 38, \infty]$

$R = [15, 24, 43, \infty]$

Now we compare the current elements.
The infinity value acts as a marker that says:

There are no more real elements in this part.

Therefore, once all real elements of one side are used, ∞ will always lose the comparison against a real number.

10 Merge Algorithm Using Infinity

The Merge procedure can be written as:

```
1 Merge(A, low, mid, high)
2
3 Create arrays L and R
4
5 Copy A[low ... mid] into L
6 Copy A[mid+1 ... high] into R
7
8 Add infinity to the end of L
9 Add infinity to the end of R
10
11 i = 0
12 j = 0
```

```
13
14 for k = low to high
15
16     if L[i] <= R[j]
17         A[k] = L[i]
18         i = i + 1
19     else
20         A[k] = R[j]
21         j = j + 1
```

The infinity values simplify the merging process because we do not need to separately check whether one array has become empty.

11 Merge Example Using Infinity

Consider:

$$L = [12, 27, 38]$$

$$R = [15, 24, 43]$$

After adding infinity:

$$L = [12, 27, 38, \infty]$$

$$R = [15, 24, 43, \infty]$$

Now compare:

$$12 < 15$$

Take 12.

$$27 > 15$$

Take 15.

$$27 > 24$$

Take 24.

$$27 < 43$$

Take 27.

$$38 < 43$$

Take 38.

Now:

$$L = \infty$$

Compare:

$$\infty > 43$$

Therefore, take 43.

Final merged array:

$$[12, 15, 24, 27, 38, 43]$$

12 Example 1: Merge Sort on Numbers

Consider:

$$A = [38, 27, 43, 3, 9, 82, 10, 15]$$

Here:

$$n = 8$$

Therefore:

$$low = 0, \quad high = 7$$

and:

$$mid = \frac{0 + 7}{2} = 3$$

Step 1: Divide

$$[38, 27, 43, 3, 9, 82, 10, 15]$$

Divide into:

$$[38, 27, 43, 3] \quad [9, 82, 10, 15]$$

Again:

$$[38, 27] \quad [43, 3]$$

$$[9, 82] \quad [10, 15]$$

Again:

$$[38] \quad [27] \quad [43] \quad [3]$$

$$[9] \quad [82] \quad [10] \quad [15]$$

Now every part contains one element.

Step 2: Merge

Merge:

$[38] + [27]$

gives:

$[27, 38]$

Merge:

$[43] + [3]$

gives:

$[3, 43]$

Merge:

$[9] + [82]$

gives:

$[9, 82]$

Merge:

$[10] + [15]$

gives:

$[10, 15]$

Now merge:

$[27, 38] + [3, 43]$

Result:

$[3, 27, 38, 43]$

And:

$[9, 82] + [10, 15]$

Result:

$[9, 10, 15, 82]$

Finally:

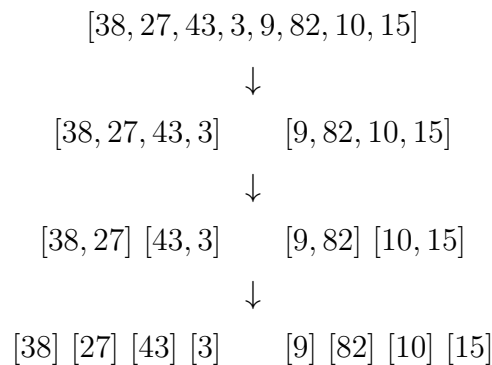
$[3, 27, 38, 43] + [9, 10, 15, 82]$

gives:

$[3, 9, 10, 15, 27, 38, 43, 82]$

13 Complete Number Example

The complete process can be represented as:



Now merge upward:

[27, 38] [3, 43] [9, 82] [10, 15]

Then:

[3, 27, 38, 43] [9, 10, 15, 82]

Finally:

[3, 9, 10, 15, 27, 38, 43, 82]

14 Example 2: Merge Sort on Alphabets

Merge Sort can also be applied to characters.

Consider:

$A = [G, C, A, F, B, E, H, D]$

We want to arrange them in alphabetical order.

Step 1: Divide

Initial:

$[G, C, A, F, B, E, H, D]$

Divide:

$[G, C, A, F] \quad [B, E, H, D]$

Again:

$[G, C] \quad [A, F]$

$[B, E] \quad [H, D]$

Again:

$[G] \quad [C] \quad [A] \quad [F]$ $[B] \quad [E] \quad [H] \quad [D]$ **Step 2: Merge**

Merge:

 $[G] + [C]$

gives:

 $[C, G]$

Merge:

 $[A] + [F]$

gives:

 $[A, F]$

Merge:

 $[B] + [E]$

gives:

 $[B, E]$

Merge:

 $[H] + [D]$

gives:

 $[D, H]$

Now:

 $[C, G] + [A, F]$

gives:

 $[A, C, F, G]$

And:

 $[B, E] + [D, H]$

gives:

 $[B, D, E, H]$

Finally:

$$[A, C, F, G] + [B, D, E, H]$$

gives:

$$\boxed{[A, B, C, D, E, F, G, H]}$$

15 Number Example vs Alphabet Example

The same Merge Sort procedure works for numbers and characters.

	Numbers	Alphabets
Input	[38, 27, 43, 3]	[G, C, A, F]
Comparison	$27 < 38$	$C < G$
Ordering	Numerical	Alphabetical
Output	[3, 27, 38, 43]	[A, C, F, G]

The algorithm remains exactly the same.

Only the comparison operation changes.

16 Time Complexity of Merge Sort

At every level, the array is divided into smaller parts.

The number of levels is approximately:

$$\log_2 n$$

At each level, the total merging work is:

$$O(n)$$

Therefore:

$$\text{Total time} = O(n) \times O(\log n)$$

Hence:

$$\boxed{O(n \log n)}$$

17 Recurrence Relation

Merge Sort can also be represented using a recurrence relation.

The array is divided into two parts.

Therefore:

$$2T\left(\frac{n}{2}\right)$$

is required to sort the two halves.

The merging requires:

$$O(n)$$

work.

Therefore:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Solving this recurrence gives:

$$O(n \log n)$$